

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Actuator
Environment
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width, self.height` → Grid dimensions.
`self.walls` → Obstacles in the grid.
`self.food_positions` → Locations of food items.
`self.opponents` → Adversarial agents in the environment.
`self.agent_pos` → Starting position of the agent.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent
Because, in a single-agent system, only one agent interacts with a passive environment. Here, opponents actively move and can collide with the agent, influencing outcomes. That makes this a multi-agent environment where agents compete for resources (food, survival).

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything an agent has ever perceived from its environment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors because, it gathers information about the agent's position, opponent positions, food detection, wall collisions, and score

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

It is Partially Observable because, The agent cannot directly perceive the entire environment state at once; it only gets partial, localized percepts.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When `execute_action()` deducts points for hitting a wall, the Performance Measure is being updated. because the score reflects how well the agent is performing relative to its goals (avoiding walls, collecting food, surviving opponents).

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because,

1. AI agents are judged by observable outcomes in the environment, not by how much computation they perform internally.
2. This ensures the agent's success is tied to its ability to interact effectively with the world (e.g., avoiding obstacles, collecting food).
3. If performance were based on internal effort, it would not reflect the agent's actual effectiveness in achieving its goals within the environment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`_init_`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Successfully adding `self.toxic_traps` but hiding them from the agent's sensors specifically alters the environment's classification from fully observable to partially observable, since the agent cannot directly perceive all relevant state information and must infer or learn about hidden hazards through experience.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `'smells_toxin'` enables the agent to act with rationality by avoiding traps, maximizing expected utility, and aligning its behavior with the performance measure (higher score and survival).

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit demonstrated that if performance measures reward the wrong behavior (like toggling the vacuum instead of cleaning), agents will exploit it. By adding a strong trap penalty, you prevent similar exploitation in your grid world, ensuring rational agents respect hazards and pursue genuine progress toward survival and food collection.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING